

HANDOVER

Product Specification • Architecture • Workflow • MVP • Play Store Plan

1. Product Vision

Handover helps people create a secure, verifiable record whenever they give an item or document to another person or organization, and track it until it is returned.

Core value: **Create** → **Share** → **Receive** → **Verify** → **Track** → **Return**.

2. Problem

- People hand over documents, devices, keys, equipment, or other items and later need to remember who received them, why, when, and when they should come back.
- The MVP creates a clear digital record and a confirmation of physical receipt.
- The product is not a marketplace, delivery service, payment app, or social network.

3. Example Use Cases

Scenario	Handover record
College	Original certificate → College Office → verification → expected return
Repair	Laptop → Repair Shop → screen repair → expected return
Friend	Power bank → Friend → temporary use → expected return

4. MVP Features

- Authentication: register, login, logout, session management
- Profile: name, email, phone, avatar
- Create/edit/cancel handover
- Handover details and history
- QR generation for verification
- QR scanning and receiver confirmation
- Return confirmation
- Due/overdue status
- Local reminders
- Supabase Row Level Security and private storage
- Professional loading, empty, success and error states
- Dark mode

5. Status Model

Main: PENDING → RECEIVED → RETURNED

Exceptional: CANCELLED, DISPUTED, EXPIRED

6. Recommended Technology Stack

Layer	Technology	Why
Mobile	Flutter + Dart	Single Android codebase; familiar and Play Store ready
UI	Material 3	Consistent, professional Android UI
State	Provider	Already familiar; simple and sufficient for this MVP
Navigation	GoRouter	Centralized routes and auth redirects
Backend	Supabase	Auth, PostgreSQL, Storage, RLS and Realtime
Database	PostgreSQL	Relational model fits handovers and events
Storage	Supabase Storage	Private item/document photos
QR	mobile_scanner + qr_flutter	Scan and generate QR codes
Notifications	flutter_local_notifications	Simple local reminders for MVP
Architecture	Provider → Repository → Service → Supabase	Separation of concerns and testability
Version control	Git + GitHub	History, backup and portfolio
Design	Figma	Design screens before implementation

7. Application Architecture

UI → Provider → Repository → Service → Supabase

- Provider: manages UI state, loading, errors and user actions.
- Repository: exposes application-level data operations without coupling the UI to Supabase.
- Service: performs Supabase Auth, database and storage operations.
- Supabase: authentication, PostgreSQL, storage, RLS and optional realtime.

8. Project Structure

```
lib/  
■■■ main.dart  
■■■ app/  
■ ■■■ app.dart  
■ ■■■ router.dart  
■ ■■■ theme.dart  
■■■ core/  
■ ■■■ constants/  
■ ■■■ errors/  
■ ■■■ utils/  
■ ■■■ widgets/  
■■■ models/  
■ ■■■ user_profile.dart  
■ ■■■ handover.dart  
■ ■■■ handover_event.dart  
■■■ services/  
■ ■■■ auth_service.dart  
■ ■■■ handover_service.dart  
■ ■■■ storage_service.dart  
■ ■■■ qr_service.dart  
■ ■■■ notification_service.dart  
■■■ repositories/  
■ ■■■ auth_repository.dart  
■ ■■■ handover_repository.dart  
■■■ providers/  
■ ■■■ auth_provider.dart  
■ ■■■ handover_provider.dart  
■ ■■■ profile_provider.dart  
■■■ features/  
■■■ auth/  
■■■ home/  
■■■ handovers/  
■■■ qr/  
■■■ profile/
```

9. Database Design

Table	Important fields
profiles	id, full_name, email, phone, avatar_url, created_at, updated_at

Table	Important fields
handovers	id, owner_id, recipient_name, recipient_email, recipient_phone, item_type, item_name, description, photo_url, purpose
handover_events	id, handover_id, event_type, performed_by, metadata, created_at

10. QR Verification Workflow

1. Sender creates handover
2. App generates secure token/QR
3. Receiver scans QR
4. Backend validates token and authorization
5. Receiver sees limited handover information
6. Receiver confirms physical receipt
7. Status changes to RECEIVED
8. Event is recorded in handover_events
9. Later, item is marked RETURNED

Never place passwords, private documents, or unnecessary personal information directly inside a QR code.

11. Security

- Supabase Auth for identity.
- RLS so users can only access authorized records.
- Private Storage bucket for sensitive photos/documents.
- Secure, non-sensitive QR tokens.
- Validate every status transition server-side.
- Do not commit .env or database credentials to GitHub.

12. Screen Map

Splash → Welcome → Login → Register → Home → Handovers → Create Handover → Handover Details → Show QR → Scan QR → Verification → Profile

13. Development Roadmap

Phase 1 — Foundation: Flutter, Material 3, Provider, GoRouter, Supabase, architecture, theme

Phase 2 — Authentication and profile

Phase 3 — Handover CRUD and history

Phase 4 — QR generation, scanning and receiver verification

Phase 5 — Return workflow and audit events

Phase 6 — Due/overdue reminders

Phase 7 — RLS, storage policies, validation and security

Phase 8 — Unit/widget/integration testing and release build

Phase 9 — Play Store assets, privacy policy, data safety, testing and production release

14. What Not to Build in MVP

Payments • Chat • Marketplace • AI • Maps • Delivery • Social feed • Subscriptions • Complex organization management • Government integrations • Blockchain

15. Quality Gate Before Play Store

- ✓ flutter analyze
- ✓ flutter test
- ✓ integration tests for core flows
- ✓ release AAB build
- ✓ test on real Android device
- ✓ privacy policy and Data Safety form
- ✓ internal/closed testing before production